INF 2020 : Travail noté 1

Marc-André Jean

Résumé

Ce rapport compare Python et C# à travers trois tâches de programmation, chacune implémentée dans les deux langages :

1. extraire la moyenne, la médiane, le minimum et le maximum de deux colonnes d'un fichier CSV d'un million de lignes;
2. trouver tous les nombres premiers inférieurs ou égaux à un nombre N (crible d'Ératosthène);
3. compter le nombre de mots du texte visible d'une page Web.

Chaque tâche sollicite un type de travail différent : traitement de données, calcul mathématique et réseau. Le temps d'exécution sert de mesure, mais l'analyse compare aussi la lisibilité, la robustesse et les bibliothèques disponibles. Toutes les mesures ont été prises le 19 octobre 2026 sur une seule machine (AMD Ryzen 7 3700X, 32 Go de RAM, Windows 11, Python 3.13, .NET 10). Les programmes en C# sont compilés en configuration Release avec `dotnet build -c Release` et lancés avec `dotnet run -c Release --no-build`.

Deux durées sont relevées pour chaque programme. Dans les deux cas, il s'agit d'un temps écoulé (durée réelle) et non d'un temps processeur :

- le **temps de traitement** est mesuré dans le programme avec un compteur de performance haute résolution (`time.perf_counter` en Python (Python Software Foundation, s.d.-d) et `Stopwatch` en C# (Microsoft, s.d.-e)). Il couvre l'exécution de la tâche, du début à la fin de la fonction principale, et exclut le démarrage de l'interpréteur Python ou de l'environnement d'exécution .NET;
- le **temps total** est mesuré de l'extérieur avec `Measure-Command` en PowerShell, du lancement de la commande jusqu'à la fin du processus. Il inclut donc le démarrage du programme.

Chaque programme a été exécuté 11 fois. La première exécution, qui sert de *réchauffage* (cache du système, du réseau, etc.), est écartée, et les moyennes présentées portent sur les 10 exécutions suivantes.

Tâche 1 : Statistiques de ventes

Choix de la tâche

La tâche consiste à lire un fichier CSV et à en extraire la moyenne, la médiane, le minimum et le maximum de deux colonnes. Elle a été choisie parce qu'elle représente un besoin très courant en analyse de données : lire un fichier tabulaire, convertir des valeurs et calculer des statistiques descriptives. Avec un million de lignes, le volume est assez grand pour que le temps d'exécution soit mesurable. La tâche permet aussi de comparer les outils standard des deux langages : les modules `csv` et `statistics` en Python, et `LINQ` (Language Integrated Query) en C#.

Le CSV de test est un fichier avec les en-têtes `id`, `category`, `price` et `quantity`. Il contient exactement 1 million d'entrées. Il a été généré avec le code Python suivant, dont le générateur aléatoire (avec un *seed*

prédéfinie) produit un fichier identique à chaque génération :

```
"""Generate sales.csv : 1 000 000 entries."""
import csv
import random
from pathlib import Path

NUMBER_OF_LINES = 1_000_000
CATEGORIES = ["food", "books", "electronic", "clothing", "garden"]

random.seed(42)
path = Path(__file__).parent / "sales.csv"

with open(path, "w", newline="", encoding="utf-8") as f:
    writer = csv.writer(f)
    writer.writerow(["id", "category", "price", "quantity"])
    for i in range(1, NUMBER_OF_LINES + 1):
        writer.writerow([
            i,
            random.choice(CATEGORIES),
            f"{random.uniform(1, 500):.2f}",  # price in dollars, 2 decimal places
            random.randint(1, 20),
        ])

print(f"{NUMBER_OF_LINES} lines written to {path}")
```

Solution en Python

Le programme lit le fichier avec `csv.DictReader`, qui accède aux colonnes par leur nom, convertit les deux colonnes numériques, puis calcule les statistiques avec le module `statistics` et les fonctions intégrées `min` et `max`.

```
"""sales_stats_from_csv.py (Task 1)
============
Output statistics (mean, median, min, max) from a CSV file.
Csv file must have a header with at least the columns "price" and "quantity".

Usage :
    python sales_stats_from_csv.py <sales.csv>
"""

import csv
import statistics
import sys
import time
from decimal import Decimal, InvalidOperation

PRICE_COLUMN = "price"
QUANTITY_COLUMN = "quantity"
```

```
def main(csv_path) -> None:
    """Output statistics (mean, median, min, max) from a CSV file.

    Args:
        csv_path: CSV filepath with columns "price" and "quantity".
    """
    # Start measuring time.
    perf_start_time = time.perf_counter()

    # Load data from CSV file.
    price_list = []
    quantity_list = []
    try:
        with open(csv_path, encoding="utf-8") as file:
            csv_data = csv.DictReader(file)
            for line in csv_data:
                price_list.append(Decimal(line[PRICE_COLUMN]))
                quantity_list.append(int(line[QUANTITY_COLUMN]))
    except (KeyError, ValueError, InvalidOperation) as e:
        print(f"Error while parsing file '{csv_path}': {e}")
        return

    # Print statistics for price and quantity.
    print("Price statistics:")
    print_stats_from_list(price_list)

    print("Quantity statistics:")
    print_stats_from_list(quantity_list)

    perf_time = (time.perf_counter() - perf_start_time) * 1000
    print(f"Execution time = {perf_time:.2f} ms")


def print_stats_from_list(list) -> None:
    """Print statistics (mean, median, min, max) from a list of values.

    Args:
        list: A list of numeric values.
    """
    print(f"\tMean:\t{statistics.mean(list):.2f}")
    print(f"\tMedian:\t{statistics.median(list):.2f}")
    print(f"\tMin:\t{min(list):.2f}")
    print(f"\tMax:\t{max(list):.2f}")


if __name__ == "__main__":
    if len(sys.argv) < 2:
        print("Error: Missing CSV file path argument.")
        sys.exit(1)
    main(sys.argv[1])
```

Solution en C#

Le programme suit la même structure. LINQ sert à la lecture (`Skip`, `Select`) et aux statistiques (`Average`, `Min`, `Max`). La médiane est calculée à la main après un tri avec `OrderBy`.

```
/* Task 1: Sales Statistics
Output statistics (mean, median, min, max) from a CSV file.
Csv file must have a header with at least the columns "price" and "quantity".
Usage : dotnet run <sales.csv>
*/
using System.Diagnostics;
using System.Globalization;

class Program
{
    const string PriceColumn = "price";
    const string QuantityColumn = "quantity";

    static void Main(string[] args)
    {
        if (args.Length < 1) {
            Console.WriteLine("Error: Missing CSV file path argument.");
            return;
        }

        // Start measuring time.
        var perfStopWatch = Stopwatch.StartNew();

        // Load data from CSV file.
        var priceList = Array.Empty<decimal>();
        var quantityList = Array.Empty<decimal>();
        try {
            var lines = File.ReadLines(args[0]).ToArray();
            var header = lines[0].Split(',').Select(h => h.Trim()).ToList();
            var priceIndex = header.IndexOf(PriceColumn);
            var quantityIndex = header.IndexOf(QuantityColumn);
            if (priceIndex < 0 || quantityIndex < 0) {
                throw new FormatException(
                    $"Missing column \"{PriceColumn}\" or \"{QuantityColumn}\" in
header.");
            }

            var csvData = lines.Skip(1).Select(l => l.Split(',')).ToArray();
            priceList = csvData
                .Select(c => decimal.Parse(c[priceIndex],
CultureInfo.InvariantCulture))
                .ToArray();
            quantityList = csvData
                .Select(c => decimal.Parse(c[quantityIndex],
CultureInfo.InvariantCulture))
                .ToArray();
```

```
        }
        catch (Exception e) {
            Console.WriteLine($"Error while parsing file '{args[0]}':
{e.Message}");
            return;
        }

        // Print statistics for price and quantity.
        Console.WriteLine("Price statistics:");
        Program.Output(priceList);

        Console.WriteLine("Quantity statistics:");
        Program.Output(quantityList);

        var perfTime = perfStopWatch.Elapsed.TotalMilliseconds;
        Console.WriteLine($"Execution time = {perfTime:F2} ms");
    }

    static void Output(decimal[] array)
    {
        var sorted = array.OrderBy(x => x).ToArray();
        var n = sorted.Length;

        var medianIndex = (int)(n / 2m);
        var median = (n % 2 != 0)
            ? sorted[medianIndex]
            : (sorted[medianIndex] + sorted[medianIndex - 1]) / 2m;

        Console.WriteLine($"\tMean:\t{array.Average():F2}");
        Console.WriteLine($"\tMedian:\t{median:F2}");
        Console.WriteLine($"\tMin:\t{array.Min():F2}");
        Console.WriteLine($"\tMax:\t{array.Max():F2}");
    }
}
```

Résultats

Les deux programmes affichent exactement les mêmes statistiques.

```
❯ python .\sales_stats_from_csv.py .\sales.csv
Price statistics:
        Mean:   250.26
        Median: 250.25
        Min:    1.00
        Max:    500.00
Quantity statistics:
        Mean:   10.50
        Median: 11.00
        Min:    1.00
        Max:    20.00
```

```
Execution time = 3811.72 ms
❯ Measure-Command { python .\sales_stats_from_csv.py .\sales.csv } | Select-Object
TotalMilliseconds
TotalMilliseconds
-----------------
          4172.23
❯ dotnet run -c Release --no-build .\sales.csv
Price statistics:
        Mean:   250.26
        Median: 250.25
        Min:    1.00
        Max:    500.00
Quantity statistics:
        Mean:   10.50
        Median: 11.00
        Min:    1.00
        Max:    20.00
Execution time = 1364.59 ms
❯ Measure-Command { dotnet run -c Release --no-build .\sales.csv } | Select-Object
TotalMilliseconds
TotalMilliseconds
-----------------
          2096.51
```

Moyenne sur 10 exécutions

Programme	Moyenne du temps de traitement	Moyenne du temps total
Python	3831.66 ms	4168.08 ms
C#	1361.44 ms	2109.10 ms

Discussion

Mise en oeuvre

Les deux programmes utilisent uniquement les bibliothèques standard de leur langage, sans paquet `pip` ni `NuGet`. Ils suivent la même structure (validation de l'argument, lecture du fichier, calcul des statistiques, affichage) et interceptent les erreurs de format (colonne absente, valeur non numérique). Les implémentations diffèrent toutefois selon les outils que chaque bibliothèque standard offre :

- **Lecture du CSV.** Python offre le module `csv`, dont `DictReader` accède aux colonnes par leur nom. C# n'a pas d'équivalent : le programme lit l'en-tête pour trouver l'index des colonnes `price` et `quantity`, puis découpe chaque ligne avec `Split(',')`. Cette approche est plus simple, mais elle ne gère pas une virgule à l'intérieur d'un champ entre guillemets, alors que le module `csv` la gère. Un paquet comme `CsvHelper` aurait donné un code C# plus lisible et plus robuste, mais il n'a pas été utilisé afin de comparer les bibliothèques standard des deux langages.
- **Médiane.** Le module `statistics` de Python fournit `median` (Python Software Foundation, s.d.-b). LINQ n'a pas d'équivalent : en C#, la médiane a été calculée à la main (tri avec `OrderBy`, puis traitement du cas d'un nombre pair de valeurs).

- **Types numériques.** Les prix sont des montants financiers. Les types à virgule flottante binaire ne représentent pas toujours exactement les nombres à décimales, et les erreurs d'arrondi peuvent s'accumuler dans les calculs (Python Software Foundation, s.d.-a; Microsoft, s.d.-b). Les deux programmes utilisent donc un type décimal : la classe `Decimal` du module `decimal` en Python, qu'il faut importer, et le type `decimal` en C#, intégré au langage. Dans les deux cas, la colonne des prix est convertie avec ce type; la colonne `quantity`, elle, est convertie en `int` en Python et en `decimal` en C#.
- **Conversion des nombres (portabilité).** En C#, `decimal.Parse` sans culture explicite utilise celle de Windows : un ordinateur configuré en français attend la virgule comme séparateur décimal, et un ordinateur configuré aux États-Unis, le point. Préciser une culture force son utilisation dans tout le programme. En Python, `Decimal(...)` n'accepte que le point : une valeur comme `"1,5"` lève une exception `InvalidOperation` (Python Software Foundation, s.d.-a), qui est interceptée par le programme.

Défis rencontrés

Aucun défi majeur n'a été rencontré.

Performance

En temps de traitement, C# est environ 2.8 fois plus rapide que Python. En temps total, l'avantage tombe à environ 2.0 fois.

Tâche 2 : Nombres premiers inférieurs ou égaux à N

Choix de la tâche

Le crible d'Ératosthène est un algorithme classique de recherche des nombres premiers. Il raye les multiples de chaque nombre premier à partir de son carré, ce qui limite la boucle externe à la racine carrée de N, pour une complexité de O(N log log N) (cp-algorithms, s.d.).

Cette tâche a été choisie parce qu'elle est du calcul pur, sans fichier ni réseau. Elle isole le coût des boucles et de la mémoire, et elle permet de comparer un même algorithme écrit avec des boucles imbriquées dans un langage interprété (Python) et dans un langage compilé (C#).

Solution en Python

```
"""prime-numbers.py (Task 2)
============
Output list of prime numbers up to a given limit.
Implementation of the `Sieve of Eratosthenes` algorithm.

Usage :
    python prime-numbers.py <limit>
"""

import math
import sys
import time
```

```
def main(n) -> None:
    """Output list of prime numbers up to a given limit.

    Args:
        n: The upper limit (inclusive) for generating prime numbers.
    """
    try:
        n = int(n)
    except ValueError:
        print("Error: Please provide an integer greater than 1.")
        return

    if n < 2:
        print("Error: Please provide an integer greater than 1.")
        return

    # Start measuring time (performance counter)
    perf_start_time = time.perf_counter()

    is_prime_list = [True] * (n + 1)
    is_prime_list[0] = is_prime_list[1] = False

    for i in range(2, int(math.sqrt(n)) + 1):
        if is_prime_list[i]:
            # We eleminate multiples of `i`.
            for j in range(i * i, n + 1, i):
                is_prime_list[j] = False

    # Print the list of prime numbers.
    primes = [i for i, is_prime in enumerate(is_prime_list) if is_prime]
    print(f"Prime numbers up to {n}:")
    print(", ".join(map(str, primes)))

    perf_time = (time.perf_counter() - perf_start_time) * 1000
    print(f"Execution time = {perf_time:.2f} ms")


if __name__ == "__main__":
    if len(sys.argv) < 2:
        print("Error: Missing limit argument.")
        sys.exit(1)
    main(sys.argv[1])
```

Solution en C#

```
/* Task 2: Prime Numbers
Output list of prime numbers up to a given limit.
Implementation of the `Sieve of Eratosthenes` algorithm.
```

```
Usage : dotnet run <limit>
*/
using System.Diagnostics;
using System.Globalization;

class Program
{
    static void Main(string[] args)
    {
        if (args.Length < 1) {
            Console.WriteLine("Error: Missing limit argument.");
            return;
        }

        if (!int.TryParse(args[0], out int n)) {
            Console.WriteLine("Error: Limit must be an integer greater than 1");
            return;
        }

        if (n < 2) {
            Console.WriteLine("Error: Limit must be an integer greater than 1.");
            return;
        }

        // Start measuring time.
        var perfStopWatch = Stopwatch.StartNew();

        var isPrimeList = Enumerable.Repeat(true, n + 1).ToArray();
        isPrimeList[0] = isPrimeList[1] = false;

        var limit = (int)Math.Sqrt(n);
        for (var i = 2; i <= limit; i++) {
            if (isPrimeList[i]) {
                for (var j = i * i; j <= n; j += i) {
                    isPrimeList[j] = false;
                }
            }
        }

        // Print the list of prime numbers.
        var primes = isPrimeList
            .Select((value, index) => new { value, index })
            .Where(x => x.value)
            .Select(x => x.index)
            .ToList();
        Console.WriteLine($"Prime numbers up to {n}:");
        Console.WriteLine(string.Join(", ", primes));

        var perfTime = perfStopWatch.Elapsed.TotalMilliseconds;
        Console.WriteLine($"Execution time = {perfTime:F2} ms");
    }
}
```

Résultats

Pour N = 10 000 000, les deux programmes affichent la même liste de 664 579 nombres premiers, raccourcie ci-dessous :

```
❯ python .\prime-numbers.py 10_000_000
Prime numbers up to 10000000:
[...]
Execution time = 1109.97 ms
❯ Measure-Command { python .\prime-numbers.py 10_000_000 } | Select-Object
TotalMilliseconds
TotalMilliseconds
-----------------
          1468.99
❯ dotnet run -c Release --no-build 10000000
Prime numbers up to 10000000:
[...]
Execution time = 209.08 ms
❯ Measure-Command { dotnet run -c Release --no-build 10000000 } | Select-Object
TotalMilliseconds
TotalMilliseconds
-----------------
           939.51
```

Moyenne sur 10 exécutions

Programme	Moyenne du temps de traitement	Moyenne du temps total
Python	1123.48 ms	1505.36 ms
C#	204.34 ms	970.61 ms

Discussion

Mise en oeuvre

Les deux programmes suivent le même algorithme avec les bibliothèques standard de leur langage, sans paquet `pip` ni NuGet. Ils valident l'argument reçu (absent, non entier ou inférieur à 2). Le crible utilise un tableau de booléens (`bool[]` en C#, une liste en Python), et les boucles sont presque identiques. Quelques différences méritent d'être notées :

- **Construction de la liste des nombres premiers.** Python crée la liste des nombres premiers en regardant la position de ceux qui sont vrais. C# utilise une chaîne LINQ (`Select`, `Where`, `Select`) qui crée un objet anonyme pour chacune des 10 millions de cases du tableau.
- **Mémoire.** Pour N = 10 000 000, un `bool[]` en C# occupe 1 octet par case, soit environ 10 Mo (Microsoft, s.d.-d). Une liste Python stocke une référence de 8 octets par case, soit environ 80 Mo selon `sys.getsizeof` (Python Software Foundation, s.d.-c). Cette différence de 8 fois peut contribuer à l'écart

de performance. Il est à noter qu'un `bytearray` aurait potentiellement pu être utilisé pour optimiser l'algorithme en Python.

Défis rencontrés

Aucun défi particulier n'a été rencontré : le crible d'Ératosthène est simple à implémenter.

Performance

En temps de traitement, C# est environ 5.5 fois plus rapide que Python. En temps total, l'avantage est d'environ 1.6 fois. Le temps de traitement inclut aussi la construction et l'affichage de la liste des 664 579 nombres premiers, et pas seulement le crible. L'écart est attendu, car la tâche est faite presque uniquement de boucles.

Tâche 3 : Compteur de mots d'une page Web

Choix de la tâche

Compter les mots d'une page Web combine deux activités courantes : télécharger une ressource par HTTP, puis analyser du HTML en retirant le contenu invisible (scripts, styles). Cette tâche relève du *web scraping* (extraction automatique de données d'un site), à distinguer du *crawling* (exploration de pages de lien en lien), qui n'est pas nécessaire ici puisqu'une seule page est visée.

Contrairement aux deux autres tâches, elle dépend du réseau. La page de test, « The longest text ever » (Rainbow Fluffy Sheep, s.d.), contient un très long texte, ce qui donne une charge d'analyse non négligeable.

Solution en Python

Le programme télécharge la page avec `urllib`, de la bibliothèque standard, puis l'analyse avec `BeautifulSoup` (Richardson, s.d.), installé avec `pip`.

```
"""scraping.py (Task 3)
============
Scrape text content from a webpage.
Requirements:
    pip install beautifulsoup4

Usage:
    python scraping.py <url>

Example:
    python scraping.py https://www.scrapethissite.com/pages/
"""

import sys
import time
from urllib.request import HTTPError, Request, URLError, urlopen

from bs4 import BeautifulSoup
```

```
def main(url) -> None:
    """Scrape text content from a webpage.

    Args:
        url: The URL of the webpage to scrape.
    """
    print("url:", url)
    # Start measuring time.
    perf_start_time = time.perf_counter()
    try:
        request = Request(url, headers={"User-Agent": "scraping/1.0"})
        html = urlopen(request).read().decode("utf-8")
    except (URLError, HTTPError) as e:
        print(f"Error while fetching URL '{url}': {e}")
        return

    soup = BeautifulSoup(html, "html.parser")

    # Remove content that isn't normally visible to the user.
    for element in soup(["script", "style", "noscript", "template"]):
        element.decompose()

    text = soup.get_text(separator=" ", strip=True)
    print("Word count:", len(text.split()))

    perf_time = (time.perf_counter() - perf_start_time) * 1000
    print(f"Execution time = {perf_time:.2f} ms")


if __name__ == "__main__":
    if len(sys.argv) < 2:
        print("Error: Missing URL argument.")
        sys.exit(1)
    main(sys.argv[1])
```

Solution en C#

Le programme télécharge la page avec `HttpClient`, de la bibliothèque standard .NET, puis l'analyse avec `HtmlAgilityPack` (HtmlAgilityPack, s.d.), installé avec NuGet. Les éléments invisibles sont retirés avec une requête XPath.

```
/* Task 3: Web Scraping
Scrape text content from a webpage.
Usage : dotnet run <url>
*/
using System.Diagnostics;
using System.Globalization;
using HtmlAgilityPack;
```

```
class Program
{
    static async Task Main(string[] args)
    {
        if (args.Length < 1) {
            Console.WriteLine("Error: Missing URL argument.");
            return;
        }

        Console.WriteLine($"url: {args[0]}");
        // Start measuring time.
        var perfStopWatch = Stopwatch.StartNew();

        // Load data from URL.
        var textContent = string.Empty;
        try {
            using (var client = new HttpClient()) {
                client.DefaultRequestHeaders.Add("User-Agent", "scraping/1.0");
                textContent = await client.GetStringAsync(args[0]);
            }
        }
        catch (Exception e) {
            Console.WriteLine($"Error while fetching URL '{args[0]}':
{e.Message}");
            return;
        }

        // Remove css, js and html content.
        var doc = new HtmlDocument();
        doc.LoadHtml(textContent);
        var nodes = doc.DocumentNode.SelectNodes("//script | //style | //noscript
| //template") ?? Enumerable.Empty<HtmlNode>();
        foreach (var node in nodes)
        {
            node.Remove();
        }
        // Extract text content and count words.
        var textNodes = doc.DocumentNode.SelectNodes("//text()") ??
Enumerable.Empty<HtmlNode>();
        string text = string.Join(" ", textNodes
            .Select(node => HtmlEntity.DeEntitize(node.InnerText).Trim())
            .Where(nodeText => nodeText.Length > 0));
        var wordCount = text.Split((char[]?)null,
StringSplitOptions.RemoveEmptyEntries).Length;

        Console.WriteLine($"Word count: {wordCount}");

        var perfTime = perfStopWatch.Elapsed.TotalMilliseconds;
        Console.WriteLine($"Execution time = {perfTime:F2} ms");
    }
}
```

Résultats

Les deux programmes trouvent exactement le même nombre de mots : `BeautifulSoup` et `HtmlAgilityPack` extraient le même texte visible. Les temps de traitement sont du même ordre de grandeur.

```
❯ python scraping.py https://rainbowfluffysheep.wordpress.com/the-longest-text-ever/
url: https://rainbowfluffysheep.wordpress.com/the-longest-text-ever/
Word count: 27325
Execution time = 264.76 ms
❯ Measure-Command { python scraping.py https://rainbowfluffysheep.wordpress.com/the-longest-text-ever/ } | Select-Object TotalMilliseconds
TotalMilliseconds
-----------------
           707.02
❯ dotnet run -c Release --no-build https://rainbowfluffysheep.wordpress.com/the-longest-text-ever/
url: https://rainbowfluffysheep.wordpress.com/the-longest-text-ever/
Word count: 27325
Execution time = 294.00 ms
❯ Measure-Command { dotnet run -c Release --no-build https://rainbowfluffysheep.wordpress.com/the-longest-text-ever/ } | Select-Object TotalMilliseconds
TotalMilliseconds
-----------------
          1091.97
```

Moyenne sur 10 exécutions

Programme	Moyenne du temps de traitement	Moyenne du temps total
Python	260.28 ms	712.31 ms
C#	306.01 ms	1074.64 ms

Discussion

Mise en oeuvre

Les deux programmes suivent la même démarche : validation de l'argument, téléchargement de la page, retrait des éléments non visibles (`script`, `style`, `noscript`, `template`), extraction du texte, puis comptage des mots. Les erreurs de téléchargement sont interceptées dans les deux cas. Contrairement aux tâches précédentes, il aurait été plus difficile que de seulement utiliser la bibliothèque standard et d'analyser le HTML à la main. Le programme C# utilise donc le paquet `HtmlAgilityPack`, et le programme Python `BeautifulSoup`. Les implémentations diffèrent sur trois points :

- **Entités HTML.** `BeautifulSoup` retourne un texte déjà décodé (par exemple `&` devient `&`). Avec `HtmlAgilityPack`, `InnerText` conserve les entités, et le programme C# doit appeler `HtmlEntity.DeEntitize` pour les décoder.

- **Modèle d'exécution.** `urllib` est synchrone : le programme attend la réponse avant de continuer, ce qui est simple. `HttpClient` en C# est asynchrone avec `async/await` (Microsoft, s.d.-a). Pour télécharger plusieurs pages, ce modèle permet de lancer les requêtes en parallèle. Python offre aussi `async/await` avec le module `asyncio`, mais `urllib` est bloquant.
- **Encodage.** Le programme Python décode toujours la page en UTF-8 (`decode("utf-8")`). `GetStringAsync` utilise le jeu de caractères annoncé dans l'en-tête `Content-Type`, puis détecte une marque d'ordre des octets (BOM), et utilise UTF-8 en dernier recours (dotnet, s.d., ligne 133). Cette approche est plus robuste pour des pages qui ne sont pas en UTF-8.

Défis rencontrés

- **En-tête `User-Agent`.** Un en-tête `User-Agent` était demandé pour récupérer la page. L'en-tête `"User-Agent": "scraping/1.0"` a donc été ajouté dans les deux programmes.
- **Scraping éthique.** Le fichier `robots.txt` du site n'interdit pas l'accès à cette page aux robots (Rainbow Fluffy Sheep, 2026), ce qui la rend appropriée pour un scraping éthique.
- **Compte de mots réel.** Obtenir le vrai nombre de mots a demandé de retirer du document les balises HTML, le CSS, le JavaScript et tous les éléments qui ne sont pas du texte visible.

Performance

En temps de traitement, Python est environ 1.2 fois plus rapide que C# (260.28 ms contre 306.01 ms). Cette mesure inclut le téléchargement de la page, qui dépend du réseau et du serveur, et les deux séries ont été mesurées l'une après l'autre : l'état du réseau a pu changer entre les deux. En temps total, Python est environ 1.5 fois plus rapide. L'écart y est plus grand que dans le temps de traitement : la différence se trouve dans ce qui n'est pas mesuré à l'intérieur du programme, soit le démarrage ou le traitement du document HTML.

Analyse comparative : Python vs C#

Aucun des deux langages ne domine sur tous les critères. Python réduit l'effort d'écriture et offre plus d'outils prêts à l'emploi; C# exécute plus vite le code écrit à la main et vérifie davantage de choses avant l'exécution.

Lisibilité et facilité de mise en oeuvre

Python demande moins de code lorsque sa bibliothèque standard couvre le besoin : à la tâche 1, la lecture du CSV par nom de colonne et le calcul de la médiane sont fournis, alors que le programme C# doit les écrire à la main.

LINQ rend C# expressif pour les calculs sur des collections (`Average`, `Min` et `Max` à la tâche 1) et il est peut rendre plus lisible la manipulation de listes (`IEnumerable`) lorsqu'il est bien utilisé.

Python n'exige aucune compilation : le script se lance directement. En C#, il faut compiler avec `dotnet build`, une étape simple, et les paquets externes s'installent facilement avec NuGet. Notez qu'on peut aussi simplement exécuter `dotnet run`, qui s'occupe également de la compilation.

Performance et mémoire

C# a été nettement plus rapide en temps de traitement aux tâches 1 et 2, qui reposent sur du calcul et du traitement de données (environ 2.8 et 5.5 fois), alors que Python a été un peu plus rapide à la tâche 3, qui

dépend du réseau et du traitement d'un fichier HTML (environ 1.2 fois). Cet avantage s'explique par le mode d'exécution : le compilateur C# produit du code intermédiaire (CIL), que le compilateur à la volée (JIT) traduit en code machine pendant l'exécution (Microsoft, s.d.-c), alors que Python exécute chaque tour de boucle avec son interpréteur. Le tableau de booléens du crible occupe aussi moins de mémoire en C# (tâche 2).

Typage et robustesse

C# est statiquement typé : une erreur de type, comme l'addition d'un nombre et d'un objet incompatible, est détectée à la compilation. Python est typé dynamiquement : chaque valeur a un type, et une opération incompatible est refusée, mais seulement à l'exécution. Les annotations de type sont facultatives; l'interpréteur ne les vérifie pas et seuls des outils externes, comme les vérificateurs de types, les utilisent (Python Software Foundation, s.d.-e).

Ce filet de sécurité ne protège toutefois que contre les erreurs de type. Une donnée invalide, comme un prix qui n'est pas un nombre, n'est détectée qu'à l'exécution dans les deux langages (`ValueError` ou `InvalidOperation` en Python, `FormatException` en C#). Dans les deux cas, la robustesse dépend d'abord des choix du programmeur.

Limites des mesures

Les résultats valent pour les programmes écrits et pour la machine utilisée, et non pour les deux langages en général : il existe un très grand nombre de façons d'implémenter un même algorithme et il est toujours possible d'optimiser un algorithme existant. Des outils spécialisés, comme *BenchmarkDotNet* pour *.NET* et *pyperf* pour Python, permettraient des tests de performance plus rigoureux.

Cas d'utilisation

Selon le point de vue, Python peut être un bon choix lorsque la rapidité de développement et les bibliothèques comptent le plus : analyse de données (tâche 1), scripts, prototypes et extraction de données du Web (tâche 3). C# peut être intéressant pour de grandes applications, où le typage statique aide à maintenir le code, pour des services réseau concurrents grâce à son modèle asynchrone, et pour des calculs intensifs écrits directement avec des boucles (tâche 2). Il n'est toutefois pas impossible de faire dans un langage ce que l'autre fait : avec les nombreux paquets disponibles de part et d'autre, le choix dépend souvent davantage du contexte du projet et de l'équipe que d'une limite du langage.

IA générative

L'autocomplétion de Copilot (GitHub, 2026) a été utilisée dans Visual Studio Code. ChatGPT-6 Luna (OpenAI, 2026) a été utilisé pour réviser et corriger les fautes de ce texte.

Requête à ChatGPT-6 Luna:

```
Mon objectif est de créer deux algorithmes (un en Python et le second en C#) pour
les tâches suivantes :
1. Extraire la moyenne, la médiane, le minimum et le maximum d'un fichier CSV ;
2. Trouver les nombres premiers égaux ou inférieurs à un nombre ;
3. Compter le nombre de mots dans un site web.
```

```
Peux-tu me donner une liste de liens et de sources qui pourraient être pertinents
dans mon cas? Trouve-en aussi sur le benchmarking. Fais une liste d'environ 15
sites web provenant de sources réputées.

Donne seulement les URL sans explication.
```

Réponse :

```
1. [https://docs.python.org/3/](https://docs.python.org/3/?utm_source=chatgpt.com)
2. [https://learn.microsoft.com/fr-ca/dotnet/csharp/]
(https://learn.microsoft.com/fr-ca/dotnet/csharp/?utm_source=chatgpt.com)
3. [https://docs.python.org/3/library/statistics.html]
(https://docs.python.org/3/library/statistics.html?utm_source=chatgpt.com)
4. [https://learn.microsoft.com/fr-ca/dotnet/csharp/linq/]
(https://learn.microsoft.com/fr-ca/dotnet/csharp/linq/?utm_source=chatgpt.com)
5. [https://learn.microsoft.com/fr-ca/dotnet/standard/io/]
(https://learn.microsoft.com/fr-ca/dotnet/standard/io/?utm_source=chatgpt.com)
6. [https://docs.python.org/3/library/csv.html]
(https://docs.python.org/3/library/csv.html?utm_source=chatgpt.com)
7. [https://opendsa-server.cs.vt.edu/](https://opendsa-server.cs.vt.edu/?
utm_source=chatgpt.com)
8. [https://docs.python.org/3/library/urllib.html]
(https://docs.python.org/3/library/urllib.html?utm_source=chatgpt.com)
9. [https://learn.microsoft.com/fr-fr/dotnet/api/system.net.http.httpclient]
(https://learn.microsoft.com/fr-fr/dotnet/api/system.net.http.httpclient?
utm_source=chatgpt.com)
10. [https://www.crummy.com/software/BeautifulSoup/bs4/doc/]
(https://www.crummy.com/software/BeautifulSoup/bs4/doc/?utm_source=chatgpt.com)
11. [https://benchmarkdotnet.org/articles/overview.html]
(https://benchmarkdotnet.org/articles/overview.html?utm_source=chatgpt.com)
12. [https://pyperf.readthedocs.io/en/stable/]
(https://pyperf.readthedocs.io/en/stable/?utm_source=chatgpt.com)
13. [https://pyperformance.readthedocs.io/](https://pyperformance.readthedocs.io/?
utm_source=chatgpt.com)
14. [https://www.nist.gov/programs-projects/nistsematech-engineering-statistics-
handbook](https://www.nist.gov/programs-projects/nistsematech-engineering-
statistics-handbook?utm_source=chatgpt.com)
15. [https://www.bigocheatsheet.com/](https://www.bigocheatsheet.com/?
utm_source=chatgpt.com)
```

Références

cp-algorithms. (s.d.). *Sieve of Eratosthenes*. https://cp-algorithms.com/algebra/sieve-of-eratosthenes.html

dotnet. (s.d.). *HttpContent.cs* [Code source]. GitHub. https://github.com/dotnet/runtime/blob/main/src/libraries/System.Net.Http/src/System/Net/Http/HttpContent.cs#L133

GitHub. (2026). *GitHub Copilot* [Logiciel]. https://github.com/features/copilot

HtmlAgilityPack. (s.d.). *Html Agility Pack*. https://html-agility-pack.net

Microsoft. (s.d.-a). *Asynchronous programming scenarios*. https://learn.microsoft.com/dotnet/csharp/asynchronous-programming/async-scenarios

Microsoft. (s.d.-b). *Floating-point numeric types (C# reference)*. https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/builtin-types/floating-point-numeric-types#characteristics-of-the-floating-point-types

Microsoft. (s.d.-c). *Managed execution process*. https://learn.microsoft.com/en-us/dotnet/standard/managed-execution-process

Microsoft. (s.d.-d). *sizeof operator - determine the storage needs for a type*. https://learn.microsoft.com/en-us/dotnet/csharp/language-reference/operators/sizeof

Microsoft. (s.d.-e). *Stopwatch, classe (System.Diagnostics)*. https://learn.microsoft.com/fr-fr/dotnet/api/system.diagnostics.stopwatch

OpenAI. (2026). *ChatGPT (ChatGPT-6 Luna)* [Grand modèle de langage]. https://chatgpt.com

Python Software Foundation. (s.d.-a). *decimal — Decimal fixed-point and floating-point arithmetic*. https://docs.python.org/3/library/decimal.html#decimal-objects

Python Software Foundation. (s.d.-b). *statistics — Mathematical statistics functions*. https://docs.python.org/3/library/statistics.html

Python Software Foundation. (s.d.-c). *sys — System-specific parameters and functions*. https://docs.python.org/3/library/sys.html#sys.getsizeof

Python Software Foundation. (s.d.-d). *time — Time access and conversions*. https://docs.python.org/3/library/time.html#time.perf_counter

Python Software Foundation. (s.d.-e). *typing — Support for type hints*. https://docs.python.org/3/library/typing.html

Rainbow Fluffy Sheep. (s.d.). *The longest text ever* [Page de blogue]. https://rainbowfluffysheep.wordpress.com/the-longest-text-ever/

Rainbow Fluffy Sheep. (2026). *robots.txt* [Fichier]. https://rainbowfluffysheep.wordpress.com/robots.txt

Richardson, L. (s.d.). *Beautiful Soup documentation*. https://www.crummy.com/software/BeautifulSoup/bs4/doc/